```
import re
```

```
# Sample Paragraph
text = """
Ravi went to the library to borrow a book.
He found a book about artificial intelligence.
The book was very interesting, so he decided to read it at home.
Ravi took the book and returned to his room.
There, he started reading it immediately.
"""

# 1. Split Paragraph into Sentences
sentences = re.split(r'(?<=[.!?])\s+', text.strip())

print("DISCOURSE STRUCTURE ANALYSIS")
print("=" * 50)

for i, sentence in enumerate(sentences, 1):
    print(f"Sentence {i}: {sentence}")


# 2. Simple Entity Extraction

pronouns = {
    "he", "she", "it", "they", "them",
    "his", "her", "their", "there"
}

stop_words = {
    "the", "a", "an", "to", "and", "or", "but", "so",
    "was", "is", "are", "were", "very", "in", "on",
    "at", "of", "for", "with", "he", "she", "it",
    "they", "them", "his", "her", "their", "there",
    "went", "found", "decided", "read", "took",
    "returned", "started", "borrow", "about"
}

person_names = {"ravi"}

locations = {"library", "home", "room"}

objects = {"book"}


def get_entities(sentence):
    words = re.findall(r'\b[A-Za-z]+\b', sentence)
    entities = []

    for word in words:
        lower_word = word.lower()

        if lower_word in person_names:
            entities.append((word, "PERSON"))

        elif lower_word in locations:
            entities.append((word, "LOCATION"))

        elif lower_word in objects:
            entities.append((word, "OBJECT"))

        elif word[0].isupper() and lower_word not in stop_words:
            entities.append((word, "PERSON"))

    return entities


# 3. Reference Resolution

print("\nREFERENCE RESOLUTION")
print("=" * 50)

previous_entities = []

for i, sentence in enumerate(sentences, 1):
    words = re.findall(r'\b[A-Za-z]+\b', sentence)
```

```
    current_entities = get_entities(sentence)

    for word in words:
        lower_word = word.lower()

        # He / His -> PERSON
        if lower_word in {"he", "his"}:
            person_candidates = [
                entity for entity, entity_type in previous_entities
                if entity_type == "PERSON"
            ]

            if person_candidates:
                antecedent = person_candidates[-1]
                print(f"Sentence {i}: '{word}' -> '{antecedent}'")
            else:
                print(f"Sentence {i}: '{word}' -> No antecedent found")

        # She / Her -> PERSON
        elif lower_word in {"she", "her"}:
            person_candidates = [
                entity for entity, entity_type in previous_entities
                if entity_type == "PERSON"
            ]

            if person_candidates:
                antecedent = person_candidates[-1]
                print(f"Sentence {i}: '{word}' -> '{antecedent}'")
            else:
                print(f"Sentence {i}: '{word}' -> No antecedent found")

        # It -> OBJECT
        elif lower_word == "it":
            object_candidates = [
                entity for entity, entity_type in previous_entities
                if entity_type == "OBJECT"
            ]

            if object_candidates:
                antecedent = object_candidates[-1]
                print(f"Sentence {i}: '{word}' -> '{antecedent}'")
            else:
                print(f"Sentence {i}: '{word}' -> No antecedent found")

        # There -> LOCATION
        elif lower_word == "there":
            location_candidates = [
                entity for entity, entity_type in previous_entities
                if entity_type == "LOCATION"
            ]

            if location_candidates:
                antecedent = location_candidates[-1]
                print(f"Sentence {i}: '{word}' -> '{antecedent}'")
            else:
                print(f"Sentence {i}: '{word}' -> No antecedent found")

        # They / Them / Their
        elif lower_word in {"they", "them", "their"}:
            if previous_entities:
                antecedent = previous_entities[-1][0]
                print(f"Sentence {i}: '{word}' -> '{antecedent}'")
            else:
                print(f"Sentence {i}: '{word}' -> No antecedent found")

    # Update previous entities
    previous_entities.extend(current_entities)


# 4. Discourse Coherence Analysis

print("\nDISCOURSE COHERENCE ANALYSIS")
print("=" * 50)

coherence_markers = {
    "because", "therefore", "however", "so",
```

```
    "then", "also", "but", "thus",
    "there", "immediately"
}

common_words = {
    "a", "an", "the", "to", "and", "or", "but",
    "so", "was", "is", "are", "were", "very",
    "in", "on", "at", "of", "for", "with",
    "he", "she", "it", "they", "them",
    "his", "her", "their", "there"
}

for i in range(len(sentences) - 1):

    current_words = set(
        re.findall(r'\b[A-Za-z]+\b', sentences[i].lower())
    )

    next_words = set(
        re.findall(r'\b[A-Za-z]+\b', sentences[i + 1].lower())
    )

    # Find shared concepts
    shared_concepts = (
        current_words.intersection(next_words)
        - common_words
    )

    # Check discourse markers
    marker_found = None

    for marker in coherence_markers:
        if marker in next_words:
            marker_found = marker
            break

    print(f"\nSentence {i + 1} -> Sentence {i + 2}")

    if shared_concepts:
        print("Shared concepts:", ", ".join(sorted(shared_concepts)))

    if marker_found:
        print("Discourse marker:", marker_found)

    if shared_concepts or marker_found:
        print("Coherence: HIGH")
    else:
        print("Coherence: LOW")


# 5. Overall Discourse Analysis

print("\nOVERALL DISCOURSE ANALYSIS")
print("=" * 50)

print("Number of sentences:", len(sentences))

print("The paragraph contains:")
print("- A sequence of related events")
print("- Pronoun references to previously mentioned entities")
print("- Repeated references to Ravi and book")
print("- Temporal and causal relationships")
print("- Overall discourse coherence: HIGH")
```

```
DISCOURSE STRUCTURE ANALYSIS
==================================================
Sentence 1: Ravi went to the library to borrow a book.
Sentence 2: He found a book about artificial intelligence.
Sentence 3: The book was very interesting, so he decided to read it at home.
Sentence 4: Ravi took the book and returned to his room.
Sentence 5: There, he started reading it immediately.

REFERENCE RESOLUTION
==================================================
Sentence 2: 'He' -> 'Ravi'
Sentence 3: 'he' -> 'Ravi'
Sentence 3: 'it' -> 'book'
Sentence 4: 'his' -> 'Ravi'
```

```
Sentence 5: 'There' -> 'room'
Sentence 5: 'he' -> 'Ravi'
Sentence 5: 'it' -> 'book'

DISCOURSE COHERENCE ANALYSIS
==================================================

Sentence 1 -> Sentence 2
Shared concepts: book
Coherence: HIGH

Sentence 2 -> Sentence 3
Shared concepts: book
Discourse marker: so
Coherence: HIGH

Sentence 3 -> Sentence 4
Shared concepts: book
Coherence: HIGH

Sentence 4 -> Sentence 5
Discourse marker: immediately
Coherence: HIGH

OVERALL DISCOURSE ANALYSIS
==================================================
Number of sentences: 5
The paragraph contains:
- A sequence of related events
- Pronoun references to previously mentioned entities
- Repeated references to Ravi and book
- Temporal and causal relationships
- Overall discourse coherence: HIGH
```